



Instituto Politécnico Nacional

Unidad Profesional Interdisciplinaria en Ingeniería y
Tecnologías Avanzadas

Ingeniería Mecatrónica

Reporte de Trabajo Terminal 1

Control de posición de robot manipulador usando redes neuronales cuantizadas

Autor:

Mendoza Díaz Ramiro

Asesor:

Dr. Jorge Alejandro Juárez Lora

Profesores:

Fernandez Nava Cecilia
Martínez Solis Sandra

Portada con firmas

(Añadir la portada de firmas aquí)

Dedicatoria...

Agradecimientos

Agradezco a...

Índice

Resumen	VI
Abstract	VI
Palabras clave	VII
Índice de figuras	VIII
Índice de tablas	IX
Nomenclatura	X
Simbología	XI
Introducción	1
0.0.1. Multiplicación de matrices, procesamiento de señales y problemas de computación en tareas robóticas	2
0.0.2. Cuello de botella de Von Neumann	2
0.0.3. Proyección a futuro de la problemática energética	3
0.0.4. Objetivo general	4
0.0.5. Objetivos específicos TT1	4
0.0.6. Objetivos específicos TT2	5
0.0.7. Técnicas de control clásico para robots manipuladores	6
0.0.8. Transición hacia el control basado en Inteligencia Artificial (IA)	7
0.0.9. Estado del arte	8
1. Marco de Referencia	9
1.1. Marco teórico	9
1.1.1. Modelado y Control de Robots Manipuladores	9
1.1.2. Redes Neuronales Artificiales para el Control Robótico	11
1.1.3. Estrategias de Optimización de Redes Neuronales	11
1.2. Marco procedimental	16
1.2.1. Metodología mecatrónica	16
2. Diseño del sistema	18
2.1. Diseño conceptual	18
2.1.1. Necesidades - requerimientos	18
2.1.2. Arquitectura funcional (funciones)	18
2.1.3. Arquitectura física (módulos)	18
2.1.4. Propuestas solución	19
2.1.5. Validación	21
2.1.6. Selección diseño conceptual	21
2.2. Diseño detallado	21
2.2.1. Detalle módulo 1 (M_1)	21

2.2.2.	Detalle módulo 2 (M_2)	21
2.2.3.	Detalle módulo n (M_n)	21
2.2.4.	Integración sistema mecatrónico	21
3.	Implementación del sistema	22
3.1.	Implementación módulo 1 (M_1)	22
3.1.1.	Verificación (M_1)	22
3.2.	Implementación módulo 2 (M_2)	22
3.2.1.	Verificación (M_2)	22
3.3.	Implementación módulo n (M_n)	22
3.3.1.	Verificación (M_n)	22
3.4.	Integración del sistema mecatrónico	22
3.4.1.	Verificación del sistema mecatrónico	22
4.	Análisis de resultados	22
4.1.	Análisis de ingeniería	22
4.2.	Análisis de costos	22
4.3.	Análisis de valor	22
	Conclusiones	23
	Recomendaciones y trabajo a futuro	23
	Apéndices	24
	Anexos	24
4.3.1.	Recursos Estimados e Infraestructura	24
4.3.2.	Recursos Humanos	24
4.3.3.	Recursos Materiales y Económicos	24
4.3.4.	Infraestructura Requerida	26
4.3.5.	Planeación de Actividades (Cronograma General)	27
	Fuentes – Referencias	29
	Glosario	32

Resumen

La robótica moderna demanda sistemas de control cada vez más complejos para que los manipuladores ejecuten tareas aún más desarrolladas con precisión y adaptabilidad. Si bien la inteligencia artificial, y en particular las redes neuronales profundas (DNNs), ofrecen soluciones potentes, su elevado costo computacional y energético restringe su aplicación en plataformas robóticas con cómputo al filo. Este proyecto de investigación se centra en este desafío, motivado por la necesidad crítica de desarrollar controladores robóticos energéticamente eficientes y de validar cuantitativamente sus beneficios.

Se plantea el problema de cómo implementar un control de posición para un robot manipulador que sea preciso y, fundamentalmente, de bajo consumo, mediante el uso de redes neuronales cuantizadas (QNNs) desplegadas en hardware reconfigurable (FPGA). El objetivo general de este trabajo es diseñar, implementar y evaluar comparativamente el desempeño energético, la utilización de recursos de hardware y la precisión del control de un sistema basado en QNNs, contrastándolo con un control PID clásico y una red neuronal de punto flotante (FP12-NN), todos operando sobre la misma plataforma FPGA.

La propuesta de solución abarca el desarrollo de estos tres enfoques de control, la aplicación de técnicas de cuantización para generar modelos QNN con distintos niveles de precisión, y la ejecución de un plan experimental para medir y analizar las métricas de rendimiento y consumo energético. Se espera que los resultados demuestren la viabilidad y las ventajas energéticas de las QNNs, aportando una metodología y datos concretos para el diseño de futuros sistemas de control robótico energéticamente sostenibles.

Abstract

Modern robotics demands increasingly complex control systems for manipulators to perform more advanced tasks with precision and adaptability. While artificial intelligence—particularly deep neural networks (DNNs)—offers powerful solutions, their high computational and energy costs limit their use in robotic platforms with edge computing. This research project addresses this challenge, driven by the critical need to develop energy-efficient robotic controllers and to quantitatively validate their benefits.

The central problem is how to implement a position control system for a robotic manipulator that is both accurate and, fundamentally, low-power, using quantized neural networks (QNNs) deployed on reconfigurable hardware (FPGAs). The main objective of this work is to design, implement, and comparatively evaluate the energy performance, hardware resource usage, and control accuracy of a QNN-based system, benchmarking it against a classical PID controller and a floating-point neural network (FP12-NN), all running on the same FPGA platform.

The proposed solution involves developing all three control strategies, applying quantization techniques to produce QNN models with varying precision levels, and executing an experimental plan to measure and analyze performance and energy consumption metrics. The expected outcome is to demonstrate the feasibility and energy advantages of QNNs, providing both a methodology and concrete data for the design of future energy-efficient robotic control systems.

Palabras clave

Redes Neuronales Cuantizadas, Control de Robots, FPGA, Eficiencia Energética, Robótica, Computo al filo.

Índice de figuras

1.	Arquitectura Von Neumann original [1]	2
2.	Reducción de bits de entrada y de precisión en la salida por la cuantización [2]	4
3.	Diagrama simplificado de un control PID [3].	6
4.	Estructura de una red neuronal (ANN) [4].	7
5.	Comparación de una ANN y una DNN [5].	8

Índice de cuadros

Nomenclatura

(Definición de términos)

Simbología

(Definición de símbolos)

Introducción

Los robots manipuladores son componentes esenciales en la automatización industrial, la cirugía asistida, la exploración espacial y una creciente variedad de aplicaciones de servicio, gracias a su capacidad para realizar tareas de manipulación con precisión y repetibilidad [6]. El control efectivo de estos sistemas es fundamental para su desempeño. Tradicionalmente, se han empleado técnicas de control clásico, como el control Proporcional-Integral-Derivativo (PID) o el control por par calculado, que han demostrado ser robustas para tareas definidas y con modelos dinámicos precisos del robot [6]. Sin embargo, la creciente demanda de mayor autonomía, adaptabilidad a entornos no estructurados y la capacidad para manejar interacciones complejas ha impulsado la adopción de enfoques basados en Inteligencia Artificial (IA), particularmente las Redes Neuronales Artificiales (ANNs) y las Redes Neuronales Profundas (DNNs) [7, 8].

A pesar de sus ventajas, la implementación de DNNs en sistemas robóticos conlleva desafíos significativos. Estos modelos suelen ser computacionalmente intensivos, caracterizados por un gran número de parámetros y una alta densidad de operaciones aritméticas, lo que se traduce en un considerable consumo energético y una fuerte demanda de recursos de memoria y procesamiento [9, 10]. Estas exigencias son particularmente problemáticas para las plataformas robóticas autónomas, que operan con fuentes de alimentación y recursos computacionales limitados [11, 12].

El interés de este estudio surge de la necesidad de reducir estos costes computacionales y energéticos. Las Redes Neuronales Cuantizadas (QNNs), que reducen la precisión numérica de los pesos y activaciones de una red, ofrecen una vía prometedora para comprimir modelos y acelerar la inferencia, con el potencial de disminuir drásticamente el consumo energético y el uso de memoria [10, 13].

Este trabajo busca aportar conocimiento práctico y cuantitativo sobre la viabilidad y los beneficios de utilizar QNNs para el control eficiente de robots manipuladores, ofreciendo una perspectiva con un enfoque en la eficiencia energética de diferentes métodos de control de sistemas robóticos.

Enfoque mecatrónico

En este apartado se describe cómo el proyecto evalúa un sistema completo que integra la mecánica del robot, la electrónica de medición y actuación, y el algoritmo de control implementado.

Definición del problema

El control de robots es esencial para su implementación en la automatización industrial o en robótica de consumo, a medida que los robots se emplean en tareas más avanzadas o con entornos cambiantes o dinámicos, los métodos de control se vuelven más complejos y con esto, su coste computacional y energético aumentan [7, 8].

0.0.1. Multiplicación de matrices, procesamiento de señales y problemas de computación en tareas robóticas

Las multiplicaciones de matrices tienen consecuencia directa en la robótica ya que las operaciones matriciales están implicadas en todos sus aspectos, particularmente la multiplicación vector-matriz y las operaciones de multiplicación-acumulación. Las ecuaciones de la cinemática directa e inversa, el modelo dinámico del robot dado por las ecuaciones de Euler-Lagrange o Newton-Euler, así como los diferentes métodos de control pudiendo ir desde un PID con compensaciones dinámicas, hasta los basados en aprendizaje automático y redes neuronales, todos dependen del uso y operación de matrices y vectores.[9, 6]

Este tipo de operaciones se realizan repetidamente a una alta frecuencia, lo que conlleva un costo computacional significativo. La ineficiencia en la realización de estas operaciones provoca un deterioro en el rendimiento en tiempo real, y un incremento en el consumo energético, que tiene su origen en la arquitectura de los equipos de computo actuales.

0.0.2. Cuello de botella de Von Neumann

La arquitectura Von Neumann, es un tipo específico de arquitectura computacional en la que tanto los datos como las instrucciones se almacenan en una memoria común como se muestra en la figura 1 donde se detalla el uso de memoria para los datos e instrucciones.

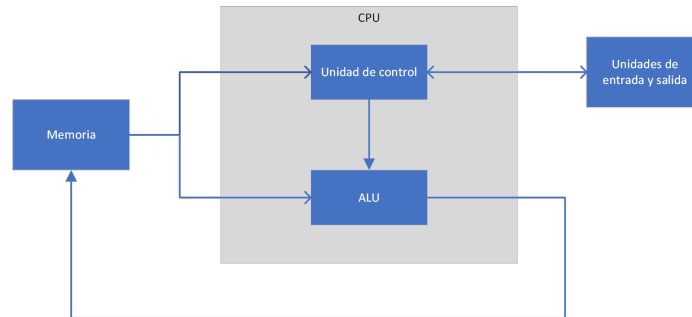


Figura 1: Arquitectura Von Neumann original [1]

Aunque este modelo ha sido fundamental para la computación digital durante muchos años, tiene limitaciones inherentes a su diseño que surgen del uso intensivo de datos, debido a la separación física entre el CPU y la memoria. Esto conlleva la necesidad de transferir datos e instrucciones a través de un bus de memoria compartido, lo que se conoce como el *cuello de botella de Von Neumann*. [1, 14]

Este cuello de botella provoca una latencia significativamente elevada, producto de la espera constante de la CPU por datos e instrucciones desde la memoria, y en un consumo energético considerable debido al intenso tráfico de datos -conocido como *memory wall*. En algoritmos con alta demanda de acceso a memoria y operaciones repetitivas sobre datos (ejemplo, multiplicación de matrices), como un control robótico,

procesamiento en tiempo real y aprendizaje automático, estas limitaciones impactan directamente en el rendimiento y la eficiencia energética del sistema.

0.0.3. Proyección a futuro de la problemática energética

Tanto la robótica como la inteligencia artificial se perfilan hacia su integración en sistemas autónomos capaces de percibir diferentes entornos, aprender de su propia experiencia y tomar decisiones en tiempo real. Debido a la complejidad de estas implementaciones, la demanda computacional ha incrementado trayendo consigo un consumo energético cada vez mayor[12, 15]. Esto presenta un problema para robots autónomos, drones, exoesqueletos de asistencia o sistemas colaborativos humano-robot donde la fuente de energía está suministrada por baterías. Este alto consumo proviene de los sistemas de control que hacen un uso intensivo de datos en arquitecturas de cómputo basadas principalmente en la arquitectura Von Neumann poniendo en juego la viabilidad de dichos sistemas autónomos.

Conforme los modelos de IA se vuelven más profundos y los sensores generan mayores volúmenes de datos, este “muro energético” se vuelve cada vez más pronunciado. Con esta proyección, se resalta la necesidad de la investigación y exploración de diferentes paradigmas computacionales que ofrezcan una mejora significativa a la eficiencia energética.

Justificación

Debido a las restricciones de cómputo actuales en chips tradicionales, surge la necesidad de mejorar su desempeño energético explorando alternativas de control enfocadas en la eficiencia computacional y energética. Este proyecto se centra en la evaluación de un método de control alternativo denominado redes neuronales cuantizadas o Quantized Neural Networks (QNNs) como una vía para lograr una mejora sustancial en la eficiencia energética de los sistemas de control de robots manipuladores. El objetivo principal es realizar una comparación directa del consumo energético de un sistema de control basado en QNNs con un control clásico bajo una plataforma de pruebas como lo puede ser una FPGA. Las redes neuronales cuantizadas reducen sus bits de entrada provocando una reducción en su requerimiento de computo y provoca una reducción también en la precisión de salida de dicha red como se ve en la figura 2[16, 17, 18].

El enfoque mecatrónico es esencial, ya que el proyecto evaluará un sistema completo que integra la mecánica del robot, la electrónica de medición y actuación, y el algoritmo de control (QNN o tradicional) implementado en una plataforma de *hardware* específica. La comparación energética se realizará considerando el sistema en su conjunto, ofreciendo una perspectiva realista del impacto de la optimización neuronal en el consumo global del robot durante tareas de control.

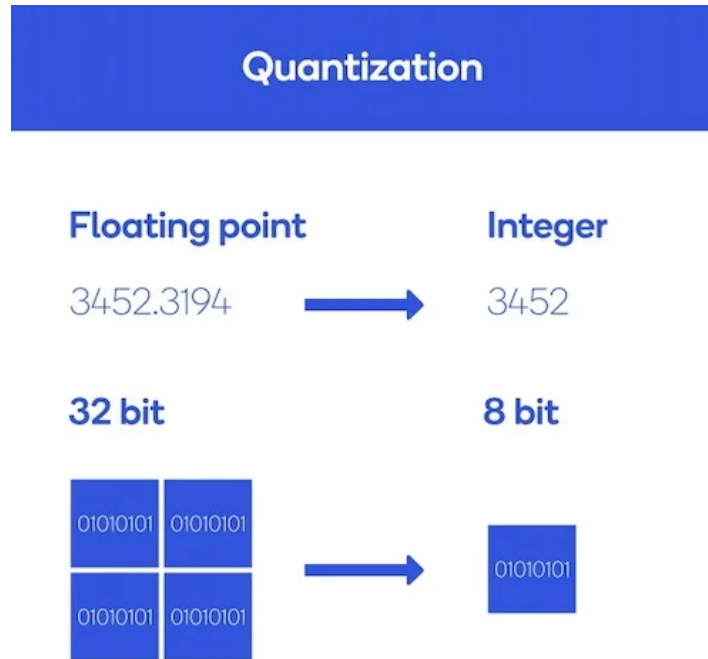


Figura 2: Reducción de bits de entrada y de precisión en la salida por la cuantización [2]

Objetivo

0.0.4. Objetivo general

Evaluar el desempeño energético, de precisión y de recursos utilizados en la implementación de un control de lazo cerrado con una planta robótica con control tradicional y con una red neuronal cuantizada.

0.0.5. Objetivos específicos TT1

- Investigar el estado del arte de las redes neuronales cuantizadas (QNNs) aplicadas al control de sistemas robóticos, las metodologías de cuantización (Post-Training Quantization - PTQ y Quantization-Aware Training - QAT), y las técnicas para la estimación y comparación del consumo energético en plataformas de cómputo embebidas y tradicionales.
- Definir la arquitectura de una red neuronal artificial (ANN) adecuada para el control de posición de un robot manipulador, incluyendo el preprocesamiento de las señales de entrada necesarias para la red.
- Diseñar el sistema de control de posición basado en la ANN seleccionada (sin cuantizar) y establecer las métricas de rendimiento de control que servirán como línea base.
- Establecer una metodología detallada para la cuantización del modelo de red neuronal diseñado, seleccionando al menos dos niveles de precisión distintos (ej.,

INT6 y INT8) y las herramientas de software para llevar a cabo el proceso.

- Diseñar el plan experimental para la comparación energética y de rendimiento, definiendo los escenarios de prueba, las variables a medir (ej., latencia de inferencia, uso de memoria, consumo energético estimado/medido, precisión del control) y la configuración del entorno de pruebas (simulado y/o físico).
- Caracterizar y poner en marcha la plataforma robótica y su interfaz de control, asegurando la capacidad de implementar y evaluar los diferentes esquemas de control.
- Escribir en el documento escrito dichos avances en la investigación.

0.0.6. Objetivos específicos TT2

- Implementar el lazo de control numérico tradicional (ej. PID) de posición del efector final, tanto en simulación numérica, como en físico, con la finalidad de tener un *benchmark* para comparar con la red neuronal cuantizada, tanto en desempeño, recurso energético, etc.
- Entrenar, implementar y validar el sistema de control de posición basado en la red neuronal no cuantizada (ANN de punto flotante) en la plataforma robótica, evaluando su rendimiento de control y su perfil de consumo energético como segundo punto de referencia.
- Implementar los sistemas de control de posición basados en las redes neuronales cuantizadas (QNNs) para los niveles de precisión seleccionados en TT1, integrándolos con la plataforma robótica.
- Realizar estudio comparativo de ambas soluciones y evaluar el desempeño energético, de control, de recursos computacionales, y de recursos de hardware implementados.
- Analizar e interpretar los resultados experimentales para cuantificar las ventajas y desventajas de cada enfoque de control, con un énfasis particular en la eficiencia energética lograda por las QNNs y su relación con la precisión del control.
- Documentar la metodología, los resultados obtenidos, las conclusiones del estudio comparativo y las recomendaciones para futuras implementaciones de QNNs en control robótico.

Antecedentes

El control de robots manipuladores es un campo bien establecido con una amplia historia de técnicas desarrolladas para lograr un movimiento preciso y fiable. Antes de la prominencia de los enfoques basados en inteligencia artificial, el control robótico se fundamentaba en gran medida en modelos matemáticos del robot y su entorno.

0.0.7. Técnicas de control clásico para robots manipuladores

Los controladores para robots manipuladores se pueden clasificar ampliamente según el espacio en el que se ejerce el control (espacio articular o espacio de la tarea) y el grado en que utilizan un modelo dinámico del robot.

Una de las estrategias más directas y comúnmente implementadas es el control PID (Proporcional-Integral-Derivativo) independiente para cada articulación. En este enfoque, cada articulación del robot es tratada como un sistema de un solo grado de libertad, y se diseña un controlador PID para regular su posición y velocidad basándose en el error entre el estado deseado y el medido (Lynch & Park, 2017, Capítulo 11.2)[6]. La simplicidad de diseño como se muestra en la figura 3 y sintonización (aunque esta última puede ser tediosa para sistemas multigrado de libertad altamente acoplados) ha hecho del control PID una opción popular, especialmente cuando no se dispone de un modelo dinámico preciso del robot. Para mejorar el rendimiento, a menudo se complementa con un control por prealimentación (*feedforward*), que utiliza el modelo cinemático y, a veces, un modelo dinámico simplificado para calcular los pares/fuerzas articulares nominales necesarios para seguir una trayectoria deseada, ayudando al lazo de realimentación PID a compensar solo las perturbaciones y errores de modelado (Lynch & Park, 2017, Capítulo 11.3)[6].

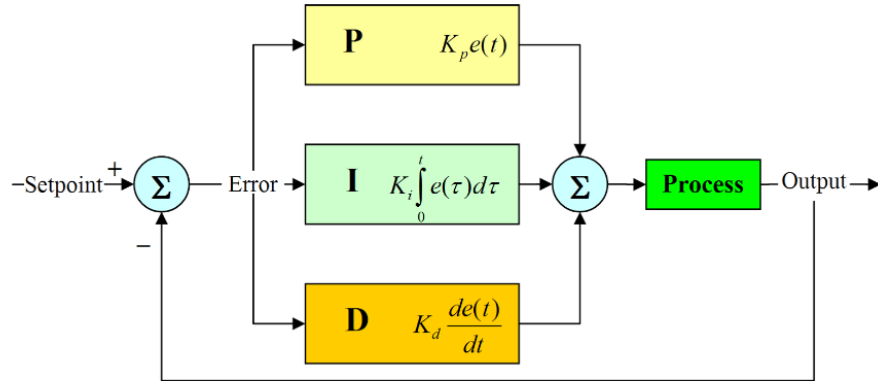


Figura 3: Diagrama simplificado de un control PID [3].

Cuando se dispone de un modelo dinámico mas completo del robot como en la ecuación 1.

$$M(\theta)\ddot{\theta} + c(\theta, \dot{\theta}) + g(\theta) = \tau \quad (1)$$

Se pueden implementar estrategias de control basadas en modelo más sofisticadas. Una técnica fundamental es el control por par calculado (*computed torque control*) o control por dinámica inversa (*inverse dynamics control*). Este enfoque busca linealizar y desacoplar la dinámica del robot mediante una ley de control que cancela los términos no lineales y de acoplamiento. La idea es calcular los pares de motor necesarios utilizando

el modelo dinámico inverso y luego aplicar un control lineal simple (como un PID) sobre el sistema linealizado resultante. Bajo la suposición de un modelo dinámico perfecto, el robot se comportaría como un sistema lineal desacoplado, lo que permitiría un control de alta precisión[6].

Otras técnicas de control avanzado incluyen el control adaptativo, que ajusta los parámetros del controlador en línea para compensar incertidumbres en el modelo dinámico del robot o variaciones en la carga útil[19], y el control robusto, diseñado para mantener la estabilidad y el rendimiento a pesar de las incertidumbres acotadas del modelo y las perturbaciones externas. El control en el espacio de la tarea (*task space control*) también es crucial, donde el control se ejerce directamente sobre la posición y orientación del efector final del robot, utilizando el jacobiano para relacionar las velocidades del efector final con las velocidades articulares.

0.0.8. Transición hacia el control basado en Inteligencia Artificial (IA)

Si bien las técnicas de control clásico mencionadas son efectivas en muchas situaciones, su rendimiento puede degradarse significativamente en presencia de dinámicas no modeladas complejas, fricción no lineal, incertidumbres paramétricas grandes o cuando se requiere que el robot opere en entornos altamente no estructurados o interactúe de forma compleja con el entorno. La obtención de modelos dinámicos precisos puede ser costosa y difícil, y la sintonización de controladores PID para sistemas multivariables fuertemente acoplados es a menudo un desafío. Estas limitaciones han motivado la exploración de enfoques basados en inteligencia artificial (IA), y en particular las redes neuronales artificiales (ANNs) de las cuales su estructura se puede apreciar en la figura 4.

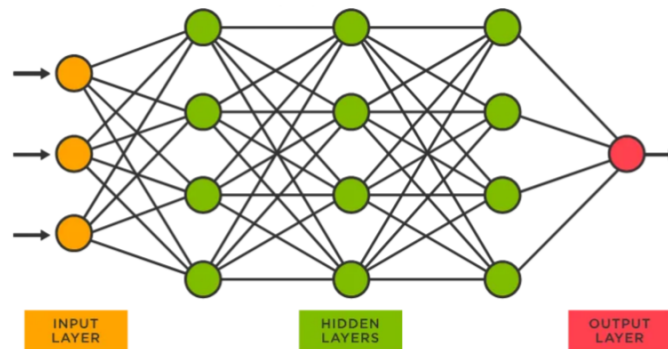


Figura 4: Estructura de una red neuronal (ANN) [4].

Las redes neuronales profundas (DNNs) mostradas en la figura 5, han emergido como herramientas poderosas para el aprendizaje de tareas de control complejas, como la cinemática inversa, el seguimiento de trayectorias y el aprendizaje por refuerzo para la manipulación[8, 20]. Estas técnicas permiten a los robots aprender de la experiencia y adaptarse a variaciones, superando en muchos casos las limitaciones de los controladores tradicionales.

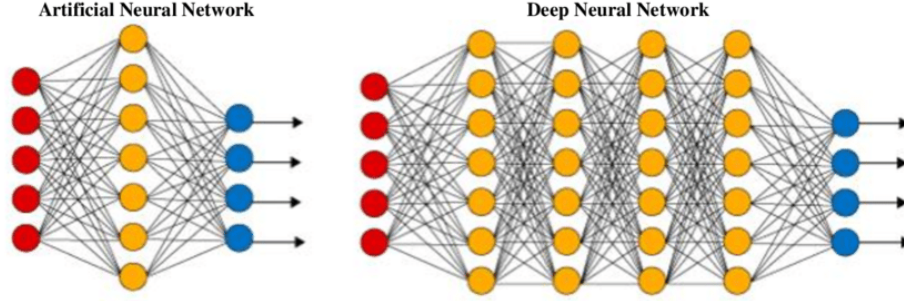


Figura 5: Comparación de una ANN y una DNN [5].

Sin embargo, el éxito y la creciente complejidad de los modelos de DNNs vienen acompañados de una demanda computacional y un consumo energético considerables[9, 12]. El gran número de parámetros y las intensivas operaciones de multiplicación y acumulación (MAC) inherentes a estas redes plantean un desafío significativo para su despliegue en plataformas robóticas, las cuales a menudo operan con recursos de hardware limitados (memoria, capacidad de procesamiento) y restricciones energéticas, especialmente en aplicaciones móviles o autónomas[11, 21]. Esta tensión entre la capacidad de los modelos de IA y las limitaciones de las plataformas de despliegue ha impulsado una vasta investigación en la optimización y compresión de redes neuronales.

0.0.9. Estado del arte

Gestionar de manera eficaz sistemas robóticos en contextos donde la energía es limitada sigue siendo un reto constante y cada vez más relevante. Aunque las arquitecturas de cómputo tradicionales, en su mayoría basadas en el esquema de Von Neumann, han facilitado importantes desarrollos, presentan limitaciones estructurales, especialmente en lo referente al acceso a la memoria y al consumo energético, cuando se utilizan en tareas de control avanzadas o en aplicaciones de inteligencia artificial. Investigaciones recientes han mostrado un interés creciente por mejorar la eficiencia de los modelos de inteligencia artificial, lo cual se refleja en iniciativas como la compresión de redes neuronales y el estudio de enfoques de cómputo alternativos a los tradicionales.

Dentro de este panorama, el cómputo neuromórfico se perfila como una opción innovadora, inspirada en la forma en que el cerebro humano procesa información de manera eficiente y paralela[22]. Este enfoque se basa en la implementación de las redes neuronales de impulsos (SNN), las cuales funcionan a través de eventos discretos (conocidos como *spikes*) y presentan una dinámica temporal inherente, lo que las hace especialmente adecuadas para aplicaciones de control en tiempo real[20, 23]. Esta implementación

de las SNN al control se ha explorado desde su aplicación al péndulo-carro [24], hasta el control de brazos robot 3D mediante aprendizaje por refuerzo (RL)[8, 20].

En estos trabajos se hace énfasis en el potencial de las QNN para aprender políticas de control con un eficiente consumo energético. Sin embargo la implementación de estas redes en *hardware*, específicamente en plataformas reconfigurables como las FPGA, sigue siendo un área de oportunidad para la investigación. El diseño en conjunto del *hardware/software* es esencial para la configuración de la arquitectura de las QNNs dentro de los recursos de una FPGA, buscando el equilibrio entre rendimiento y consumo[16].

A pesar de estos avances aun existe la necesidad de estudios comparativos que cuantifiquen la eficiencia energética de estos controles neuromórficos frente a los enfoques tradicionales. Este proyecto busca llenar este vacío proporcionando una evaluación directa en una plataforma común aplicado a un robot manipulador.

Organización del documento

En el Capítulo 1 se aborda el marco de referencia en el que se sustenta el proyecto. En el Capítulo 2 se plantea el diseño del sistema, desglosando los requerimientos, la arquitectura y las propuestas de solución. En el Capítulo 3 se detalla la implementación técnica e integración de los módulos mecatrónicos. En el Capítulo 4 se analizan los resultados obtenidos técnica y económicamente. Finalmente, se presentan las conclusiones correspondientes.

1. Marco de Referencia

1.1. Marco teórico

Este proyecto se fundamenta en la intersección de varias disciplinas clave: la robótica de manipulación, el control automático, la inteligencia artificial (específicamente las redes neuronales) y las técnicas de optimización de modelos para la eficiencia computacional y energética.

1.1.1. Modelado y Control de Robots Manipuladores

Los robots manipuladores son sistemas mecánicos articulados cuyo movimiento y comportamiento se describen mediante modelos matemáticos precisos.

Cinemática La cinemática estudia la geometría del movimiento. La **cinemática directa** relaciona las variables articulares $\mathbf{q} \in \mathbb{R}^n$ (donde n es el número de grados de libertad) con la configuración (posición y orientación) del efector final $\mathbf{x} \in SE(3)$ mediante una función $\mathbf{x} = f(\mathbf{q})$. La **cinemática inversa** busca la función $\mathbf{q} = f^{-1}(\mathbf{x})$, que es esencial para el control de posición y puede ser no lineal y tener múltiples soluciones [6].

Dinámica El modelo dinámico de un robot manipulador relaciona los pares/fuerzas articulares aplicados $\boldsymbol{\tau} \in \mathbb{R}^n$ con las posiciones $\mathbf{q}$, velocidades $\dot{\mathbf{q}}$ y aceleraciones $\ddot{\mathbf{q}}$ articulares. La ecuación general de la dinámica para un manipulador rígido es:

$$\boldsymbol{\tau} = M(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{c}(\mathbf{q}, \dot{\mathbf{q}}) + \mathbf{g}(\mathbf{q}) \quad (2)$$

Donde:

- $M(\mathbf{q}) \in \mathbb{R}^{n \times n}$ es la matriz de inercia, simétrica y definida positiva.
- $\mathbf{c}(\mathbf{q}, \dot{\mathbf{q}}) \in \mathbb{R}^n$ es el vector de pares/fuerzas centrífugas y de Coriolis.
- $\mathbf{g}(\mathbf{q}) \in \mathbb{R}^n$ es el vector de pares/fuerzas gravitacionales.

Obtener un modelo dinámico preciso es fundamental para el diseño de controladores avanzados [6].

Estrategias de Control Clásicas

- **Control PID Independiente por Articulación:** Para cada articulación i , la ley de control es:

$$\tau_i(t) = K_{p_i}e_i(t) + K_{i_i} \int_0^t e_i(\lambda)d\lambda + K_{d_i} \frac{de_i(t)}{dt} \quad (3)$$

donde $e_i(t) = q_{d_i}(t) - q_i(t)$ es el error de la articulación i , y K_p, K_i, K_d son las ganancias proporcional, integral y derivativa respectivamente. A menudo se complementa con un término de prealimentación (*feedforward*) τ_{ff_i} para mejorar el seguimiento de trayectorias[6]:

$$\tau_i = \tau_{ff_i} + K_{p_i}e_i + K_{i_i} \int e_i dt + K_{d_i} \dot{e}_i \quad (4)$$

- **Control por Par Calculado (*Computed Torque Control*):** Esta técnica utiliza el modelo dinámico para cancelar las no linealidades. Si se tiene un modelo estimado $\hat{M}(\mathbf{q})$, $\hat{\mathbf{c}}(\mathbf{q}, \dot{\mathbf{q}})$, y $\hat{\mathbf{g}}(\mathbf{q})$, la ley de control es:

$$\boldsymbol{\tau} = \hat{M}(\mathbf{q})\mathbf{a}_{cmd} + \hat{\mathbf{c}}(\mathbf{q}, \dot{\mathbf{q}}) + \hat{\mathbf{g}}(\mathbf{q}) \quad (5)$$

donde $\mathbf{a}_{cmd}$ es una aceleración de comando, comúnmente diseñada usando un control lineal sobre el error:

$$\mathbf{a}_{cmd} = \ddot{\mathbf{q}}_d + K_d(\dot{\mathbf{q}}_d - \dot{\mathbf{q}}) + K_p(\mathbf{q}_d - \mathbf{q}) \quad (6)$$

Bajo la suposición de un modelo perfecto ($\hat{M} = M, \hat{\mathbf{c}} = \mathbf{c}, \hat{\mathbf{g}} = \mathbf{g}$), el sistema en lazo cerrado se comportaría como un conjunto de n integradores dobles desacoplados: $\ddot{\mathbf{e}} + K_d\dot{\mathbf{e}} + K_p\mathbf{e} = 0$, permitiendo un control preciso [6].

1.1.2. Redes Neuronales Artificiales para el Control Robótico

Las Redes Neuronales Artificiales (ANN), y sus variantes profundas (DNN), pueden aprender mapeos complejos a partir de datos, lo que las hace adecuadas para tareas de control robótico donde los modelos analíticos son imprecisos o difíciles de obtener.

Pueden usarse para aprender la cinemática inversa $\mathbf{q} = f_{NN}(\mathbf{x}; \boldsymbol{\theta}_{NN})$, la dinámica inversa $\boldsymbol{\tau} = f_{NN}(\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}; \boldsymbol{\theta}_{NN})$, o directamente políticas de control $\boldsymbol{\tau} = \pi_{NN}(\text{estado}; \boldsymbol{\theta}_{NN})$ mediante aprendizaje por refuerzo [8, 20], donde $\boldsymbol{\theta}_{NN}$ representa los parámetros de la red.

Las operaciones fundamentales en las DNN, como las convoluciones o las capas totalmente conectadas, se basan en un gran número de operaciones de Multiplicación y Acumulación (MAC): $y = \sum_i w_i x_i + b$ para una neurona, o de forma matricial $\mathbf{Y} = \mathbf{W}\mathbf{X} + \mathbf{B}$ para una capa, lo que resulta en una alta carga computacional y accesos a memoria [9].

1.1.3. Estrategias de Optimización de Redes Neuronales

La alta demanda computacional y energética de las DNN, detallada en secciones previas, se convierte en el principal problema a tratar en este proyecto. El objetivo general es reducir la complejidad del modelo (en términos de parámetros y operaciones), el tamaño en memoria, la latencia de inferencia y el consumo energético, manteniendo al mismo tiempo la precisión de la tarea lo más alta posible. Las principales estrategias incluyen la poda de redes, el desarrollo de arquitecturas eficientes, el uso de paradigmas alternativos como las redes neuronales de impulsos, y la cuantización de redes neuronales, siendo esta última el foco central de este proyecto.

Poda de Redes (*Neural Pruning*) La poda de redes neuronales se basa en la observación de que muchos modelos de DNNs están sobreparametrizados, conteniendo un gran número de pesos o neuronas que contribuyen poco o nada al rendimiento final [10]. El objetivo de la poda es identificar y eliminar estos elementos redundantes.

Tipos de Poda:

- **Poda No Estructurada (*Unstructured Pruning*):** Consiste en eliminar pesos individuales de la red. Se establece un criterio de importancia (saliencia) para cada peso w_{ij} , y aquellos por debajo de un umbral son eliminados (su valor se fija a cero). Formalmente, si $\mathcal{T}$ es el umbral:

$$w'_{ij} = \begin{cases} w_{ij} & \text{si } \text{saliencia}(w_{ij}) > \mathcal{T} \\ 0 & \text{en caso contrario} \end{cases} \quad (7)$$

- **Poda Estructurada (*Structured Pruning*):** En lugar de eliminar pesos individuales, se eliminan grupos de pesos o estructuras enteras, como neuronas completas, filtros convolucionales completos o incluso capas. Por ejemplo, se puede calcular la importancia de un filtro convolucional (ej., sumando las magnitudes

de sus pesos o usando su norma L1/L2) y eliminar los filtros menos importantes. Esto resulta en redes más pequeñas y con una estructura regular, lo que facilita la aceleración en hardware estándar, ya que las dimensiones de las matrices y tensores se reducen directamente [10].

Proceso de Poda: La poda puede realizarse de forma iterativa: se entrena la red, se podan los elementos menos importantes, y luego se re-entrena (*fine-tuning*) la red podada para recuperar la precisión que se haya podido perder. Este ciclo de podar y re-entrenar puede repetirse varias veces. Alternativamente, existen métodos de poda *one-shot* que intentan realizar la poda en una sola etapa, a veces durante el mismo entrenamiento o inmediatamente después, como la propuesta de “*Single-shot pruning and quantization*” de Jiang [17].

Redes Neuronales de Impulsos (*Spiking Neural Networks* - SNN) Las SNN representan un paradigma de computación neuronal alternativo, inspirado más directamente en el funcionamiento de las neuronas biológicas. Se las considera la “tercera generación” de redes neuronales y se caracterizan por procesar información mediante secuencias de eventos discretos en el tiempo, denominados impulsos (*spikes*).

Modelos Neuronales de SNN: A diferencia de las neuronas de las ANN que calculan una salida analógica (o un valor de activación continuo), las neuronas en las SNN integran las señales de entrada (*spikes* ponderados de otras neuronas) a lo largo del tiempo, y generan un *spike* de salida solo cuando su potencial de membrana alcanza un umbral específico.

- ***Leaky Integrate-and-Fire* (LIF):** Es uno de los modelos más simples y computacionalmente eficientes. El potencial de membrana $v(t)$ de una neurona LIF evoluciona según la ecuación diferencial:

$$\tau_m \frac{dv(t)}{dt} = -(v(t) - v_{rest}) + R_{in}I(t) \quad (8)$$

donde τ_m es la constante de tiempo de la membrana, v_{rest} es el potencial de reposo, R_{in} es la resistencia de entrada, e $I(t)$ es la corriente sináptica total de entrada. Cuando $v(t)$ alcanza el umbral v_{th} , la neurona emite un *spike* y $v(t)$ se reinicia (ej., a $v_{reset} < v_{th}$) y puede entrar en un periodo refractario.

- **Modelo de Izhikevich:** Es un modelo bidimensional que puede reproducir una rica variedad de patrones de disparo observados en neuronas biológicas (ej., *regular spiking*, *bursting*, *chattering*, *fast spiking*, etc.) con solo cuatro parámetros ajustables (a, b, c, d) y un par de ecuaciones diferenciales acopladas:

$$\frac{dv}{dt} = 0,04v^2 + 5v + 140 - u + I \quad (9)$$

$$\frac{du}{dt} = a(bv - u) \quad (10)$$

Con la condición de reseteo post-impulso: si $v(t) \geq v_{peak}$ (usualmente 30 mV), entonces $v(t) \leftarrow c$ y $u(t) \leftarrow u(t) + d$. La variable u representa una variable de recuperación de la membrana. Este modelo es utilizado, por ejemplo, en la tesis del Dr. Juárez Lora para la implementación de una plataforma neuromórfica [7].

Las SNN ofrecen un gran potencial para la eficiencia energética, especialmente cuando se implementan en *hardware* neuromórfico dedicado (ej., Loihi de Intel, TrueNorth de IBM, SpiNNaker) que está diseñado para manejar la comunicación basada en eventos y la computación dispersa en el tiempo y el espacio [25].

Redes Neuronales Cuantizadas (*Quantized Neural Networks* - QNN) La cuantización es una técnica de compresión y optimización que reduce la precisión numérica utilizada para representar los pesos $\mathbf{w}$ y/o las activaciones $\mathbf{a}$ de una red neuronal. Esta es la estrategia de optimización central de este proyecto.

Principios Fundamentales y Proceso de Cuantización: El objetivo es transformar los parámetros y activaciones, típicamente almacenados como números de punto flotante de 32 bits (FP32), a representaciones de enteros de baja precisión (ej., INT6, INT4) o formatos de punto fijo. Una cuantización **uniforme afín** (asimétrica) de un valor real x a un entero cuantizado x_q de k bits se define mediante una escala $S \in \mathbb{R}^+$ y un punto cero $Z \in \mathbb{Z}$ (que es un entero de k bits y representa el mapeo del valor real 0,0):

$$x_q = \text{clamp} \left(\text{round} \left(\frac{x}{S} \right) + Z, N_{min}^{(k)}, N_{max}^{(k)} \right) \quad (11)$$

donde $\text{round}(\cdot)$ es el redondeo al entero más cercano, y $\text{clamp}(\cdot, N_{min}^{(k)}, N_{max}^{(k)})$ asegura que x_q permanezca dentro del rango representable por k bits (ej., $[0, 2^k - 1]$ para k bits sin signo, o $[-2^{k-1}, 2^{k-1} - 1]$ para k bits con signo). La escala S determina el tamaño del paso de cuantización, y Z permite que el rango de cuantización no sea necesariamente simétrico alrededor del cero real. La operación inversa (de-cuantización) para obtener una aproximación de x es:

$$\hat{x} = S(x_q - Z) \quad (12)$$

En la cuantización **uniforme simétrica**, el punto cero Z se asume como 0 (o el punto cero de la representación entera es el cero real), y el mapeo es simétrico alrededor del origen. La escala S se determina a partir del valor absoluto máximo del rango. La determinación de los parámetros S y Z es crucial:

- **Min-Max Quantization:** $S = \frac{\max(x_{tensor}) - \min(x_{tensor})}{N_{max}^{(k)} - N_{min}^{(k)}}$ y Z se deriva para alinear el cero.
- Otros métodos buscan minimizar el error de cuantización, por ejemplo, minimizando la divergencia KL entre la distribución de los valores FP32 y los valores cuantizados, o utilizando percentiles para ser robustos a outliers.

La cuantización puede aplicarse con diferente granularidad:

- **Por Tensor:** Un único par (S, Z) para todo un tensor (ej., todos los pesos de una capa).
- **Por Canal:** Un par (S, Z) diferente para cada canal de salida de un filtro convolucional o para cada fila/columna de una matriz de pesos. Generalmente ofrece mejor precisión que la cuantización por tensor, especialmente para capas con distribuciones de pesos muy variables entre canales.

Tipos de Datos de Baja Precisión y Formatos de Punto Fijo: Mientras que INT6 es un objetivo común y bien soportado, la investigación explora precisiones aún menores como INT4, INT2 (ternario: $\{-1, 0, 1\}$) o INT1 (binario: $\{-1, 1\}$ o $\{0, 1\}$) [26]. La representación de **punto fijo** $Q(I, F)$ denota un número con I bits para la parte entera y F bits para la parte fraccionaria (más un bit de signo si es necesario). El valor real es $X_{int} \cdot 2^{-F}$, donde X_{int} es la representación entera. La cuantización moderna a menudo prefiere el formalismo de escala y punto cero para enteros, ya que es más flexible y se adapta mejor a las capacidades del *hardware*.

Post-Training Quantization (PTQ): PTQ se aplica a un modelo ya entrenado en FP32. El proceso típicamente implica:

1. **Calibración:** Se pasa un pequeño conjunto de datos representativos (usualmente sin etiquetas, del conjunto de entrenamiento o validación) a través del modelo FP32 para recolectar estadísticas de los rangos de activación de cada capa. Los rangos de los pesos se obtienen directamente del modelo entrenado.
2. **Cálculo de Parámetros de Cuantización:** Usando los rangos observados, se calculan S y Z para cada tensor a cuantizar.
3. **Conversión del Modelo:** Los pesos se cuantizan y almacenan en el formato de baja precisión. Las activaciones se cuantizarán dinámicamente durante la inferencia, o se insertarán operaciones de cuantización/de-cuantización en el grafo.

Para mejorar la precisión de PTQ, especialmente con pocos bits, se utilizan técnicas como:

- **Bias Correction:** Se ajustan los vectores de sesgo de las capas para compensar el error medio introducido por la cuantización de los pesos, que puede desplazar la media de las activaciones.
- **Cross-Layer Equalization (CLE):** Se re-escalan los pesos entre capas consecutivas para igualar sus rangos dinámicos antes de la cuantización, lo que puede reducir el error de cuantización global. Métodos como AdaRound ajustan los pesos de forma más sofisticada para minimizar el error de redondeo [10, 27].

Quantization-Aware Training (QAT): QAT simula el proceso de cuantización durante la fase de entrenamiento (o más comúnmente, durante una fase de re-entrenamiento o *fine-tuning* a partir de un modelo FP32 pre-entrenado). Esto se logra insertando nodos de **Fake Quantization (FQ)** en el grafo computacional, típicamente después de los pesos y de las activaciones.

- **Forward Pass:** En un nodo FQ, la entrada x (FP32) se cuantiza a k bits (simulando el efecto de S y Z) y luego se de-cuantiza inmediatamente para obtener $\hat{x}$ (FP32 con valores discretos). Este $\hat{x}$ se pasa a la siguiente operación.

$$\hat{x}_{FQ} = S \left(\text{clamp} \left(\text{round} \left(\frac{x}{S} \right) + Z, N_{min}^{(k)}, N_{max}^{(k)} \right) - Z \right) \quad (13)$$

- **Backward Pass:** El gradiente de la función de pérdida con respecto a $\hat{x}_{FQ}$ debe propagarse hacia x . Dado que la función $\text{round}(\cdot)$ tiene gradiente cero casi en todas partes, se utiliza el **Straight-Through Estimator (STE)**, que simplemente copia el gradiente: $\frac{\partial L}{\partial x} = \frac{\partial L}{\partial \hat{x}_{FQ}}$. Es decir, se asume $\frac{\partial \hat{x}_{FQ}}{\partial x} \approx 1$.

QAT permite que los pesos del modelo se ajusten durante el entrenamiento para compensar la pérdida de precisión introducida por la cuantización, resultando generalmente en una mayor exactitud final en comparación con PTQ, especialmente para cuantización a muy bajos bits [10, 27].

Cuantización de Precisión Mixta (*Mixed-Precision Quantization*): No todas las capas de una red neuronal tienen la misma sensibilidad a la cuantización. Algunas pueden tolerar precisiones muy bajas (ej., INT4) con poca degradación, mientras que otras (ej., la primera o la última capa) pueden requerir mayor precisión (ej., INT6 o incluso FP16) para mantener el rendimiento general del modelo. La cuantización de precisión mixta asigna diferentes números de bits a diferentes capas o tensores, buscando un óptimo global entre compresión/eficiencia y precisión. La determinación de la política de asignación de bits es un problema complejo. Los enfoques incluyen:

- Métodos basados en análisis de sensibilidad heurístico.
- Técnicas de búsqueda automatizada como AutoML o aprendizaje por refuerzo, donde un agente aprende a asignar precisiones.
- Optimización multiobjetivo, como en MOHAQ de Rezk (2022)[27], que utiliza algoritmos genéticos para encontrar soluciones en el frente de Pareto de error vs. recursos de hardware.

Impacto en Operaciones Aritméticas y Eficiencia: La cuantización transforma las operaciones de punto flotante en operaciones enteras. Consideremos la multiplicación de matrices o convoluciones, que son sumas de productos: $y = \sum w_i x_i + b$. Si w_i y x_i son cuantizados a k_w y k_x bits respectivamente, con parámetros (S_w, Z_w) y (S_x, Z_x) :

$$S_y(y_q - Z_y) \approx \sum_i [S_w(w_{q_i} - Z_w)][S_x(x_{q_i} - Z_x)] + S_b(b_q - Z_b) \quad (14)$$

Desarrollando y agrupando las escalas:

$$y_q \approx Z_y + \frac{S_w S_x}{S_y} \sum_i (w_{q_i} - Z_w)(x_{q_i} - Z_x) + \frac{S_b}{S_y} (b_q - Z_b) \quad (15)$$

La suma $\sum_i (w_{q_i} - Z_w)(x_{q_i} - Z_x)$ involucra productos y acumulaciones de enteros, que se realizan típicamente en una precisión mayor (ej., INT32). El factor de re-escala $M = \frac{S_w S_x}{S_y}$ (y similar para el sesgo) se calcula offline y puede aproximarse por un producto de un entero y una potencia de dos para ser implementado con multiplicaciones enteras y bit-shifts, evitando operaciones de punto flotante en tiempo de inferencia. Las operaciones de normalización de batch (Batch Normalization), si presentes, a menudo se “pliegan” en los pesos y sesgos de las capas lineales o convolucionales precedentes antes de la cuantización, eliminando la necesidad de calcularlas durante la inferencia cuantizada. Los beneficios son una reducción significativa en el tamaño del modelo (ej., INT6 vs FP32 es una reducción de aproximadamente 4 veces), menor ancho de banda de memoria, y aceleración de cómputo debido a que las operaciones enteras son inherentemente más rápidas y menos costosas energéticamente que las de punto flotante en la mayoría de los procesadores y hardware dedicado.

1.2. Marco procedimental

1.2.1. Metodología mecatrónica

Para la gestión y desarrollo de este proyecto de investigación, que inherentemente combina elementos de diseño de software (controladores neuronales, cuantización), integración de hardware (plataforma robótica, sistema de cómputo) y experimentación (simulación y/o pruebas físicas), se ha optado por adoptar y adaptar el marco de trabajo ágil **Scrum**. Esta elección se fundamenta en la flexibilidad de *Scrum* para manejar la incertidumbre inherente a la investigación, su énfasis en la entrega incremental de valor y la colaboración continua, son aspectos importantes para un proyecto de con un componente significativo de desarrollo tecnológico y evaluación comparativa [28].

Scrum, originalmente se crea para el desarrollo de software, pese a eso se ha aplicado al desarrollo de proyectos de ingeniería [29]. Su naturaleza iterativa e incremental permite la validación y verificación constante del progreso, un principio fundamental en el diseño mecatrónico.

Eventos de *Scrum* Adaptados al Proyecto Los eventos de *Scrum* proporcionarán la estructura para la inspección y adaptación regular:

- ***Sprint***: Serán iteraciones de duración fija (ej., 2 a 4 semanas) durante las cuales se trabajará para completar un conjunto de objetivos del *Product Backlog* y crear un Incremento utilizable o evaluable del proyecto. La duración se definirá al inicio de cada fase (TT1 y TT2).
- **Planificación del *Sprint* (*Sprint Planning*)**: Al inicio de cada *Sprint*, se seleccionarán los ítems de mayor prioridad del *Product Backlog* que se pueden

completar en el *Sprint* y definirá un Objetivo del *Sprint* (*Sprint goal*). Se desglosarán las tareas necesarias (*Sprint Backlog*).

- **Reuniones de Avance (Adaptación del *Daily Scrum*):** Se realizarán reuniones periódicas (ej., semanales) con los asesores para inspeccionar el progreso hacia el Objetivo del *Sprint*, discutir los trabajos completados, los que están en curso, los problemas encontrados y planificar los siguientes pasos. Esto reemplaza al *Daily Scrum* en un contexto de trabajo individual supervisado.
- **Revisión del *Sprint* (*Sprint Review*):** Al final de cada *Sprint*, se presentará el Incremento logrado (ej., un módulo de software funcional, resultados de simulación, un avance en la investigación documentado) a los asesores. Se discutirá el trabajo completado y se recogerá retroalimentación que podrá influir en el *Product Backlog* y en la planificación de *Sprints* futuros.
- **Retrospectiva del *Sprint* (*Sprint Retrospective*):** Después de la Revisión del *Sprint*, se realizará una reflexión sobre el proceso seguido durante el *Sprint*: qué funcionó bien, qué se podría mejorar y qué acciones concretas se tomarán para optimizar la eficiencia y efectividad en el siguiente *Sprint*.

Artefactos de *Scrum* Adaptados al Proyecto Los artefactos de *Scrum* proporcionarán transparencia y oportunidades para la inspección y adaptación:

- ***Product Backlog*:** Será una lista ordenada y dinámica de todo lo que se necesita para el proyecto. Incluirá los objetivos específicos de TT1 y TT2, desglosados en tareas de investigación, diseño, desarrollo, experimentación y documentación.
- ***Sprint Backlog*:** Es el conjunto de ítems del *Product Backlog* seleccionados para un *Sprint*, más un plan para entregar el Incremento y alcanzar el Objetivo del *Sprint*.
- **Incremento:** Es la suma de todos los ítems del *Product Backlog* completados durante un *Sprint* y todos los *Sprints* anteriores. Al final de cada *Sprint*, el nuevo Incremento debe ser *utilizable* o **evaluable** en el contexto del proyecto, lo que significa que representa un avance tangible y verificable hacia los objetivos generales.

Esta adopción de la metodología *Scrum* proporciona una planificación estructurada pero flexible, fomentando el progreso constante, la adaptación de los hallazgos y la entrega de resultados de forma incremental.

2. Diseño del sistema

2.1. Diseño conceptual

2.1.1. Necesidades - requerimientos

2.1.2. Arquitectura funcional (funciones)

2.1.3. Arquitectura física (módulos)

Aunque los detalles finos de la arquitectura emergerán y se refinarán durante los *Sprints*, se concibe una arquitectura modular general que facilite el desarrollo incremental y la integración. Los principales componentes que se desarrollarán o integrarán iterativamente son:

- **Componente Robot Manipulador (Simulado/Físico con Interfaz Hardware):** Como se describió en la sección 2.1.4, será un robot sujeto a elección simulado en herramientas como CoppeliaSim, con una interfaz definida para recibir comandos de actuadores y enviar datos de sensores (posiciones articulares) hacia la FPGA.
- **Componente FPGA (Núcleo de Control, Medición e Interfaz):** Este es el corazón del sistema y contendrá:
 - *Sub-componente de Interfaz con Robot:* Módulos HDL para la comunicación con los sensores (ej., lectura de encoders) y actuadores (ej., generación de señales PWM) del robot.
 - *Sub-componente de Procesamiento de Control:* Bloques HDL intercambiables que implementarán la lógica de cada uno de los controladores: PID, FP12-NN (o su equivalente de alta precisión fija), y las variantes QNN (INT6, INT4). Estos bloques recibirán el estado del robot y generarán los comandos de actuación.
 - *Sub-componente de Medición de Recursos y Estimación de Energía:* Lógica para medir latencias de cómputo, y puntos de acceso para que las herramientas de Vivado estimen el uso de recursos (LUTs, BRAMs, DSPs) y la potencia consumida por cada configuración de controlador activa.
 - *Sub-componente de Comunicación con PC:* Interfaz (ej., UART, Ethernet simulada) para recibir comandos de alto nivel, parámetros de configuración, trayectorias, y para enviar datos de telemetría y resultados al PC.
- **Componente PC (Entorno de Desarrollo, Orquestación, Entrenamiento y Análisis):**
 - *Sub-componente de Diseño y Síntesis FPGA:* Uso de Xilinx Vivado para el desarrollo HDL, simulación, síntesis, implementación y generación de bitstreams.

- *Sub-componente de Diseño, Entrenamiento y Cuantización de NN*: Uso de Python con TensorFlow/PyTorch para definir arquitecturas NN, entrenar modelos FP32, y aplicar cuantización (PTQ/QAT) para generar modelos QNN.
- *Sub-componente de Orquestación de Experimentos y Análisis*: Scripts en Python para controlar el flujo de los experimentos, enviar comandos a la FPGA, recolectar datos de rendimiento y energía, procesar estos datos y generar las gráficas y tablas comparativas.

2.1.4. Propuestas solución

Módulos ($M_1 \dots M_n$) La **visión del producto** de este proyecto es desarrollar una plataforma experimental y un conjunto de resultados que demuestren cuantitativamente las ventajas y compromisos del uso de redes neuronales cuantizadas para el control de robots manipuladores, específicamente en términos de consumo energético y utilización de recursos de hardware, en comparación con controladores PID tradicionales y redes neuronales de punto flotante (FP12-NN), todos implementados y evaluados en una plataforma FPGA.

Para alcanzar esta visión, se utilizarán las siguientes **tecnologías y herramientas base** (sujetas a refinamiento durante los *Sprints* iniciales de investigación):

- **Robot Manipulador**: Se realizará la selección de un robot existente, para su caracterización y simulación con algún toolbox como el **Robotics, Vision and Control** o software como **CoppeliaSim (V-REP)**.
- **Plataforma FPGA**: Una tarjeta de desarrollo **Xilinx Artix-7** (ej., Nexys A7 o Basys 3) o una plataforma **Zynq** (ej., PYNQ-Z2) si se requiere un sistema de procesamiento ARM integrado. El diseño para la FPGA se realizará utilizando **VHDL o Verilog** y las herramientas del **Xilinx Vivado Design Suite**.
- **PC para el desarrollo**:
 - Lenguaje principal de scripting y desarrollo de alto nivel: **Python 3.x**.
 - Frameworks de Deep Learning para diseño, entrenamiento y cuantización de NNs: **TensorFlow 2.x (con TensorFlow Lite Quantization Toolkit)** o **PyTorch (con su Quantization API)**.
 - Software de comunicación y control de la FPGA desde el PC.
 - Herramientas de análisis de datos y visualización (ej., NumPy, Pandas, Matplotlib).
- **Controladores a Implementar y Comparar**:
 - Controlador Proporcional-Integral-Derivativo (PID) clásico.
 - Red Neuronal de Punto Flotante (FP12-NN) para control.
 - Redes Neuronales Cuantizadas (QNNs) con al menos dos niveles de precisión (ej., INT8 e INT6).

■ Mecanismos de Interfaz y Medición:

- Interfaz FPGA-Robot: Lógica para leer encoders simulados/reales y enviar comandos a actuadores simulados/reales.
- Interfaz FPGA-PC: Para carga de configuraciones, envío de trayectorias, y recepción de datos de telemetría y resultados.
- Sistema de estimación/medición de energía y recursos en FPGA: Utilizando reportes de Vivado (potencia estática y dinámica estimada, uso de LUTs, BRAMs, DSPs) y/o instrumentación para mediciones más directas.

Integración (sistema) El trabajo necesario para desarrollar la solución se organizará en un *Product Backlog*, que contendrá una lista priorizada de Épicos (grandes bloques de funcionalidad o investigación). Estos Épicos se descompondrán en *Product Backlog Items* (PBIs) más pequeños y manejables durante la Planificación de cada *Sprint*. A continuación, se presenta una lista preliminar de Épicos:

EP01: Configuración del Entorno de Desarrollo y Simulación Robótica. Incluye la selección final y configuración detallada del robot manipulador, el establecimiento de la comunicación entre Python y el simulador, y la definición de las tareas de control base (ej., seguimiento de trayectorias específicas).

EP02: Diseño e Implementación del Controlador PID en FPGA. Involucra el diseño del algoritmo PID, su implementación en HDL (VHDL/Verilog), simulación HDL, síntesis para la FPGA objetivo, y pruebas de control con el robot simulado y físico interactuando con la FPGA.

EP03: Diseño y Entrenamiento del Modelo de Red Neuronal de Punto Flotante (FP12-NN) para Control. Cubre la definición de la arquitectura de la red neuronal (ej., MLP para cinemática inversa), la generación o adquisición de datos de entrenamiento, el proceso de entrenamiento, y la validación de su rendimiento en simulación.

EP04: Diseño e Implementación del Controlador FP12-NN en FPGA. Implica el diseño de una arquitectura HDL para la inferencia del modelo FP12-NN, su simulación, síntesis, y pruebas de control con el robot interactuando con la FPGA. Se evaluará la viabilidad y los recursos consumidos por una implementación de alta precisión en FPGA.

EP05: Investigación, Selección y Aplicación de Técnicas de Cuantización. Cubre el estudio de las herramientas de cuantización (TensorFlow Lite, PyTorch Quantization), la selección de estrategias PTQ y/o QAT, y la generación de modelos QNN con los niveles de precisión objetivo (ej., INT8, INT6) a partir del modelo FP12-NN entrenado.

EP06: Diseño e Implementación de los Controladores QNN en FPGA. Involucra el diseño de arquitecturas HDL optimizadas para la inferencia de los modelos

QNN (aprovechando operaciones enteras, menor ancho de banda de memoria), su simulación, síntesis, y pruebas de control con el robot interactuando con la FPGA para cada nivel de cuantización.

EP07: Diseño e Implementación del Sistema de Medición/Estimación de Consumo Energético y Uso de Recursos en FPGA. Cubre el desarrollo de la lógica en HDL (si es necesaria para contadores o monitores internos) y/o la metodología para utilizar las herramientas del fabricante de la FPGA (ej., Vivado Power Estimator, reportes de utilización) para obtener datos fiables del consumo energético y los recursos lógicos (LUTs, BRAMs, DSPs) para cada controlador implementado.

EP08: Diseño e Implementación del Sistema de Medición del Rendimiento del Control. Define e implementa los mecanismos para medir la precisión del control (ej., error de seguimiento de trayectoria, tiempo de establecimiento) en la FPGA.

EP09: Desarrollo de la Interfaz de Orquestación de Pruebas y Recolección de Datos. Construcción de scripts o una interfaz simple en el PC para configurar y lanzar los experimentos comparativos de manera sistemática, enviar trayectorias a la FPGA y recolectar todos los datos de rendimiento, recursos y energía.

EP10: Ejecución de Experimentos Comparativos Exhaustivos y Análisis de Resultados. Realización de las pruebas comparativas bajo condiciones controladas para los controladores PID, FP12-NN y QNNs en la FPGA. Análisis estadístico de los datos recolectados, interpretación de los resultados y elaboración de conclusiones sobre la eficiencia energética y los compromisos de rendimiento.

2.1.5. Validación

2.1.6. Selección diseño conceptual

2.2. Diseño detallado

2.2.1. Detalle módulo 1 (M_1)

Validación (M_1)

2.2.2. Detalle módulo 2 (M_2)

Validación (M_2)

2.2.3. Detalle módulo n (M_n)

Validación (M_n)

2.2.4. Integración sistema mecatrónico

Validación sistema mecatrónico

3. Implementación del sistema

3.1. Implementación módulo 1 (M_1)

3.1.1. Verificación (M_1)

3.2. Implementación módulo 2 (M_2)

3.2.1. Verificación (M_2)

3.3. Implementación módulo n (M_n)

3.3.1. Verificación (M_n)

3.4. Integración del sistema mecatrónico

3.4.1. Verificación del sistema mecatrónico

4. Análisis de resultados

4.1. Análisis de ingeniería

4.2. Análisis de costos

4.3. Análisis de valor

Conclusiones

Recomendaciones y trabajo a futuro

Apéndices

Anexos

La gestión adecuada de los recursos y una planificación clara de las actividades son cruciales para el éxito de este proyecto. A continuación, se presenta un desglose de los recursos humanos, materiales, económicos y la infraestructura necesaria, seguido de una planeación general de actividades.

4.3.1. Recursos Estimados e Infraestructura

4.3.2. Recursos Humanos

Los participantes en el proyecto y sus roles son los siguientes:

- **Desarrollador:** Mendoza Díaz Ramiro.
 - *Dedicación Estimada:* Aproximadamente 600 horas distribuidas en las fases TT1 y TT2.
 - *Área Principal STEM de Aporte:* Ingeniería (Mecatrónica).
 - *Otras Áreas STEM de Aporte:* Tecnología, Ciencias de la Computación.
- **Asesor Principal:** Dr. Jorge Alejandro Juárez Lora.
 - *Dedicación Estimada:* Aproximadamente 60 horas de asesoría.
 - *Área Principal STEM de Aporte:* Ingeniería (Electrónica/IA).
 - *Otras Áreas STEM de Aporte:* Ciencias de la Computación.

4.3.3. Recursos Materiales y Económicos

A continuación, se listan los recursos materiales y económicos estimados para el proyecto:

- **Plataforma Robótica (Módulo 1):**
 - *Recurso:* Robot manipulador.
 - *Cantidad:* 1.
 - *Costo Estimado (MXN):* \$5,000. - \$15,000.
 - *Fuente de Financiamiento:* Recursos Propios.
 - *Recurso (Opcional):* Componentes para interfaz hardware (si se conecta simulador a FPGA mediante una placa intermedia).
 - *Cantidad:* 1.
 - *Costo Estimado (MXN):* \$500 - \$1,000.
 - *Fuente de Financiamiento:* Recursos Propios.

■ **Plataforma FPGA (Módulo 2):**

- *Recurso:* Tarjeta de desarrollo FPGA (ej., Xilinx Nexys A7 o PYNQ-Z2).
- *Cantidad:* 1.
- *Costo Estimado (MXN):* \$3,000 - \$7,000.
- *Fuente de Financiamiento:* Recursos Propios o tarjeta prestada por el asesor.
- *Recurso:* Cables y accesorios para FPGA (JTAG, USB, etc.).
- *Cantidad:* Varios.
- *Costo Estimado (MXN):* \$500.
- *Fuente de Financiamiento:* Recursos Propios.

■ **PC para Desarrollo (Módulo 3):**

- *Recurso:* Computadora personal para desarrollo (existente).
- *Cantidad:* 1.
- *Costo Estimado (MXN):* N/A (equipo existente).
- *Fuente de Financiamiento:* N/A.
- *Recurso:* Software de desarrollo (Xilinx Vivado, Python, TensorFlow/PyTorch, CoppeliaSim).
- *Cantidad:* Licencias o descargas.
- *Costo Estimado (MXN):* N/A (se priorizará software gratuito, con licencia educativa o ya disponible).
- *Fuente de Financiamiento:* N/A.

■ **Material Diverso y Consumibles (Módulo 4):**

- *Recurso:* Componentes electrónicos menores (resistencias, cables, protoboard, si se construye alguna interfaz simple).
- *Cantidad:* Varios.
- *Costo Estimado (MXN):* \$300.
- *Fuente de Financiamiento:* Recursos Propios.
- *Recurso:* Material de impresión y papelería (para documentación, artículos).
- *Cantidad:* Según necesidad.
- *Costo Estimado (MXN):* \$200.
- *Fuente de Financiamiento:* Recursos Propios.

■ **Imprevistos:**

- *Descripción:* Reserva para gastos no contemplados (aproximadamente 10 % del subtotal de gastos directos).

- *Costo Estimado (MXN)*: A definir una vez se tengan costos más precisos.
- *Fuente de Financiamiento*: Recursos Propios.

Nota: Los costos son estimaciones preliminares. Las necesidades y costos pueden variar a medida de que avanza el proyecto.

4.3.4. Infraestructura Requerida

La infraestructura necesaria para el proyecto incluye:

■ Instalaciones Físicas:

- Laboratorio de Cómputo de UPIITA: Para acceso a software especializado y desarrollo general.
- Espacio de Trabajo Personal: Para desarrollo y estudio.
- (Opcional) Laboratorio de Sistemas digitales de UPIITA: Si se requiere para pruebas con un robot físico, construcción de interfaces, o mediciones energéticas.

■ Equipo Principal:

- Computadora Personal (PC personal).
- Tarjeta de Desarrollo FPGA: Puede ser la Digilent Nexys 3 de los laboratorios de electronica de UPIITA, Intesc amiba 2 propia, o la proporcionada por el asesor.
- Robot Manipulador Físico: Pueden ser modelos desde 4 hasta 6 grados de libertad disponibles en el mercado que se ajusten al presupuesto o el proporcionado por el asesor.
- Instrumentación Electrónica: Multímetro, osciloscopio, fuente de alimentación regulada.

■ Software Especializado:

- Entorno de Desarrollo Integrado (IDE) para FPGA: Xilinx Vivado Design Suite (se utilizará la versión WebPACK gratuita o la disponible bajo licencia académica en UPIITA).
- Lenguajes de Programación: Python (versión 3.8 o superior), VHDL o Verilog (según la preferencia para el diseño en FPGA).
- Frameworks de Deep Learning: TensorFlow (versión 2.x con TensorFlow Lite) y/o PyTorch (con su módulo de cuantización).
- Simulador Robótico: CoppeliaSim (V-REP) en su versión educativa (gratuita).
- Software de Productividad: Un editor de LaTeX como overleaf, un gestor de referencias bibliográficas, software para creación de diagramas.
- Sistema de Control de Versiones: Git, con un repositorio en GitHub o GitLab.

4.3.5. Planeación de Actividades (Cronograma General)

La gestión detallada del proyecto, incluyendo la asignación de tareas específicas y el seguimiento del progreso, se realizará mediante *Sprints* de duración definida (ej., 2 a 4 semanas), conforme a la metodología *Scrum* adoptada (ver Sección 1.2.1). Para fines de este protocolo, se presenta a continuación una planeación general de alto nivel, distribuyendo los Épicos del proyecto a lo largo de las 36 semanas estimadas (18 semanas para Trabajo Terminal 1 - TT1, y 18 semanas para Trabajo Terminal 2 - TT2).

Fase TT1 (Semanas 1-18): Enfoque en Investigación, Diseño y Desarrollos Preliminares

■ *Semanas 1-4:*

EP01 Configuración del Entorno de Desarrollo y Simulación Robótica (Coppelia-Sim, Python, interfaz básica).

INV01 Investigación exhaustiva del estado del arte: QNNs, control robótico, FPGAs, metodologías de evaluación energética.

DOC1 Inicio de la documentación de avances.

■ *Semanas 5-8:*

EP03 Diseño conceptual de la arquitectura de Red Neuronal (FP12-NN) para control. Generación/adquisición de datos de entrenamiento.

EP02 Diseño conceptual del controlador PID y su adaptación para FPGA.

DOC1 Continuación de la documentación.

■ *Semanas 9-13:*

EP03 Entrenamiento y validación (simulación Python) del modelo FP12-NN.

EP02 Inicio del desarrollo HDL y simulación del controlador PID para FPGA.

EP05 Investigación detallada y selección de técnicas y herramientas de cuantización.

DOC1 Continuación de la documentación.

■ *Semanas 14-18:*

EP02 Finalización de la implementación y pruebas básicas del PID en FPGA (interactuando con robot simulado).

EP05 Aplicación preliminar de cuantización (PTQ) al modelo FP12-NN para generar un primer QNN (ej., INT6).

DOC1 Preparación y finalización del informe de avances TT1.

ENT1 Entrega y presentación de avances TT1.

Fase TT2 (Semanas 19-36): Enfoque en Implementación en FPGA, Experimentación y Análisis

■ *Semanas 19-22:*

EP04 Diseño HDL e implementación del controlador FP12-NN en FPGA. Simulación y pruebas.

EP05/EP06 Refinamiento de la cuantización (posible QAT) y diseño HDL de los controladores QNN (INT6, INT4).

DOC2 Inicio de la redacción de la tesis con toda la documentación previa.

■ *Semanas 23-26:*

EP06 Implementación y pruebas en FPGA de los controladores QNN.

EP07 Diseño e implementación del sistema de medición/estimación de consumo energético y uso de recursos en la FPGA.

DOC2 Avance en la tesis.

■ *Semanas 27-31:*

EP08 Diseño e implementación del sistema de medición del rendimiento del control (precisión, latencia) en la FPGA.

EP09 Desarrollo de la interfaz de orquestación de pruebas y recolección de datos (PC - FPGA).

EP10 Inicio de la ejecución de experimentos comparativos.

DOC2 Continuación de la tesis.

■ *Semanas 32-36:*

EP10 Finalización de los experimentos. Análisis estadístico e interpretación de todos los resultados comparativos.

DOC2 Redacción final de la tesis, incluyendo conclusiones y trabajo futuro.

ENT2 Entrega de la tesis y preparación para la presentación final.

Nota: Este cronograma es una guía general de alto nivel. La asignación específica de tareas a los Sprints y su duración se definirá dinámicamente durante el proyecto, facilitando la adaptación a los hallazgos y desafíos que surjan. INV01, DOC1, DOC2, ENT1 y ENT2 son hitos o actividades transversales.

Fuentes – Referencias

Referencias

- [1] R. Eigenmann and D. J. Lilja, “Von neumann computers,” January 1998, purdue University / University of Minnesota.
- [2] All About Circuits. (2022, March) Neural network quantization: What is it and how does it relate to tiny machine learning? Accessed: 2025-05-10. [Online]. Available: <https://www.allaboutcircuits.com/technical-articles/neural-network-quantization-what-is-it-and-how-does-it-relate-to-tiny-machine-learning/>
- [3] Wattco. (2024, julio) Controlador pid explicado: tipos, usos y operaciones. Accessed: 2025-05-01. [Online]. Available: <https://www.wattco.com/es/2024/07/controlador-pid-explicado/>
- [4] P. N. H. (2024, October) Artificial neural networks (anns) and their role in machine learning. Accessed: 2025-05-12. [Online]. Available: <https://medium.com/@prasanNH/artificial-neural-networks-anns-and-their-role-in-machine-learning-e19a214e46ec>
- [5] Q. Jaffery. (2023, September) Ann vs dnn. Accessed: 2025-05-12. [Online]. Available: <https://www.linkedin.com/pulse/ann-vs-dnn-qasim-jaffery/>
- [6] F. C. P. Kevin M. Lynch, *Modern robotics*. Reino Unido: Cambridge University Press, 2017.
- [7] J. A. J. Lora, “Design and implementation of a neuromorphic processing platform for robotic control systems,” Tesis de Doctorado en Ciencias de la Computación, Centro de Investigación en Computación, Instituto Politécnico Nacional, Ciudad de México, diciembre 2024.
- [8] Y. Park, J. Lee, D. Sim, Y. Cho, and C. Park, “Designing spiking neural network-based reinforcement learning for 3d robotic arm applications,” *Electronics*, vol. 14, no. 3, p. 578, 2025, article number 578. Check year if accessing preprint vs final version.
- [9] J. Chen, J. Li, Y. Li, and X. Miao, “Multiply accumulate operations in memristor crossbar arrays for analog computing,” *Journal of Semiconductors*, vol. 42, no. 1, p. 013104, 2021.
- [10] T. Liang, J. Glossner, L. Wang, S. Shi, and X. Zhang, “Pruning and quantization for deep neural network acceleration: A survey,” *Neurocomputing*, vol. 461, pp. 370–403, 2021.
- [11] C. Bartolozzi, G. Indiveri, and E. Donati, “Embodied neuromorphic intelligence,” *Nature Communications*, vol. 13, p. 1024, 2022.

- [12] A. de Vries-Gao, “Artificial intelligence: Supply chain constraints and energy implications,” *Joule*, vol. 9, pp. 1–4, may 2025, published April 10, 2025.
- [13] L. Zhang, X. Hu, X. Liao, T. Zhou, Y. Peng, and Q. Zhao, “FLQ: Design and implementation of hybrid multi-base full logarithmic quantization neural network acceleration architecture based on FPGA,” *Signal Processing: Image Communication*, vol. 134, p. 117270, 2025, available online 16 January 2025.
- [14] I. I. Arikpo, F. U. Ogban, and I. E. Eteng, “VON NEUMANN ARCHITECTURE AND MODERN COMPUTERS,” *GLOBAL JOURNAL OF MATHEMATICAL SCIENCES*, vol. 6, no. 2, pp. 97–103, 2007.
- [15] Álvaro Domingo Reguero, S. Martínez-Fernández, and R. Verdecchia, “Energy-efficient neural network training through runtime layer freezing, model quantization, and early stopping,” *Computer Standards & Interfaces*, vol. 92, p. 103906, 2025, published online 10 October 2024.
- [16] Y. Zhang, B. Yin, M. D. Gomony, H. Corporaal, C. Trinitis, and F. Corradi, “Hardware/software co-design optimization for training recurrent neural networks at the edge,” *Journal of Low Power Electronics and Applications*, vol. 15, no. 1, p. 15, 2025, article number 15. Check year if accessing preprint vs final version.
- [17] B. Jiang, J. Chen, and Y. Liu, “Single-shot pruning and quantization for hardware-friendly neural network acceleration,” *Engineering Applications of Artificial Intelligence*, vol. 126, p. 106816, 2023.
- [18] Y. S. Lee, Y.-H. Gong, S. W. Chung, Y. Kim, and E. Lim, “Scale-CIM: Precision-scalable computing-in-memory for energy-efficient quantized neural networks,” *Journal of Systems Architecture*, vol. 134, p. 102787, 2023, available online 30 November 2022.
- [19] K. M. Lynch and F. C. Park, “Robot control,” in *Modern Robotics*, L. Miller and K. Roberts, Eds. Berlin: Cambridge University Press, 2017, pp. 434–437.
- [20] L. Zanatta, F. Barchi, S. Manoni, S. Tolu, A. Bartolini, and A. Acquaviva, “Exploring spiking neural networks for deep reinforcement learning in robotic tasks,” *Scientific Reports*, vol. 14, p. 30648, 2024, article number 30648. Published online: 28 December 2024. Check year if different in final print.
- [21] D. Kudithipudi, C. Schuman, C. M. Vineyard, T. Pandit, C. Merkel, R. Kubendran, J. B. Aimone, G. Orchard, C. Mayr, R. Benosman, J. Hays, C. Young, C. Bartolozzi, A. Majumdar, S. G. Cardwell, M. Payvand, S. Buckley, S. Kulkarni, H. A. Gonzalez, G. Cauwenberghs, C. S. Thakur, A. Subramoney, and S. Furber, “Neuromorphic computing at scale,” *Nature*, vol. 637, pp. 801–812, 2025, published online: 22 January 2025. Check year if accessing preprint vs final version.
- [22] A. Mehonic and A. J. Kenyon, “Brain-inspired computing needs a master plan,” *Nature*, vol. 604, no. 7905, pp. 255–260, 2022.

- [23] W. Maass, “Networks of spiking neurons: The third generation of neural network models,” *Neural Networks*, vol. 10, no. 9, pp. 1659–1671, 1997.
- [24] D. Vlasov, A. Minnekhanov, R. Rybka, Y. Davydov, A. Sboev, A. Serenko, A. Ilyasov, and V. Demin, “Memristor-based spiking neural network with online reinforcement learning,” *Neural Networks*, vol. 166, pp. 512–523, 2023.
- [25] Intel. (2021) Llevar la computación neuromórfica al siguiente nivel con loihi 2. Accessed: 2025-05-01. [Online]. Available: <https://www.intel.la/content/www/xl/es/research/neuromorphic-computing-loihi-2-technology-brief.html>
- [26] X. Li, H. Jiang, R. Zhang, F. Tian, S. Huang, and D. Xu, “Robustness-aware 2-bit quantization with real-time performance for neural network,” *Neurocomputing*, vol. 455, pp. 12–22, 2021.
- [27] N. M. Rezk, T. Nordström, D. Stathis, Z. Ul-Abdin, E. E. Aksoy, and A. Heman, “MOHAQ: Multi-Objective Hardware-Aware quantization of recurrent neural networks,” *Journal of Systems Architecture*, vol. 133, p. 102778, dec 2022.
- [28] K. Schwaber and J. Sutherland, *The Scrum Guide™*, November 2020, the Definitive Guide to Scrum: The Rules of the Game. Available at <https://scrumguides.org>. [Online]. Available: <https://scrumguides.org>
- [29] E. L. Cano, J. M. García-Camús, J. Garzás, J. M. Moguerza, and N. N. Sánchez, “A scrum-based framework for new product development in the non-software industry,” *Journal of Engineering and Technology Management*, vol. 61, p. 101634, 2021. [Online]. Available: <https://doi.org/10.1016/j.jengtecman.2021.101634>

Glosario